TankmanPro — Product Requirements Document

Greenfield home-services booking platform with city-wise vendor routing

Product & Engineering Specification

20 October 2026

Table of Contents

TankmanPro

Product Requirements Document (PRD)

Version: 1.0
Status: Build specification
Build environment: Google AI Studio Build mode
Product type: Responsive, full-stack web application for Indian home services
Primary currency: INR (₹)
Primary market: India; launch cities and serviceable PIN codes are configurable by Admin.

Product direction: Create a clean, modern home-services marketplace inspired by the usability patterns of leading service-booking platforms. Do not copy Urban Company's brand, exact screens, text, logos, illustrations, or proprietary assets. TankmanPro must have its own identity and original UI.

1. Executive summary

TankmanPro is a home-services booking platform. Customers discover and book services such as water tank cleaning, AC service/repair, washing machine repair, refrigerator service, plumbing and other Admin-configured services. Customers select their city or enter a six-digit Indian PIN code before availability and prices are shown.

The platform has three distinct experiences:

Customer storefront and account: browse services, check area coverage, compare/see prices, choose a slot, apply a valid coupon, pay or choose an enabled offline option, and track bookings.

Main Admin console: manage cities, PIN codes, services, categories, service variants, add-ons, city prices, vendor-specific prices, coupons, vendors, bookings, payment/refund states, content and reports.

Vendor portal: each vendor has a separate login and only sees bookings assigned to that vendor. Vendors can accept/reject assigned jobs and update permitted work statuses, but cannot browse other vendors' orders, edit customer prices, issue refunds, or access the Admin console.

The app must be genuinely full-stack. Buttons, forms, filters, login, data writes and status changes must work against persistent data and authorized backend endpoints. Do not deliver a static UI prototype disguised as a working app.

2. Product goals and success measures

2.1 Goals

Launch a fresh website and system rather than continuing the deleted prototype.

Deliver a premium, clean, white/light, mobile-first customer experience.

Make service availability and pricing depend on configured city/PIN code, service and vendor coverage.

Let Admin manage day-to-day business data without editing code.

Isolate Vendor permissions so vendors only access their own assigned bookings.

Support controlled automatic assignment and Admin manual reassignment.

Support coupon rules, payment verification, booking status tracking and notifications.

Keep the architecture understandable and maintainable when Google AI Studio generates the first version.

2.2 Initial success measures

These are launch targets, not claims about current performance:

A customer can reach the booking confirmation from the homepage using a tested end-to-end flow.

Admin can create or deactivate a service, city, PIN code, vendor and vendor-service mapping without code changes.

Admin can configure city-specific and vendor-specific prices and see the effective price before publishing.

Every order has a server-validated location, service, immutable price snapshot and explicit assignment state.

Unauthorized cross-vendor order reads/updates fail at both the API/backend and database rules layer.

Layout works at 360 px mobile width, tablet widths and desktop widths without horizontal overflow.

No live payment is marked paid based only on a frontend redirect or client-supplied value.

The public storefront does not display Admin/Vendor portal links or role-switchers.

Production hostnames render the correct app shell: tankmanpro.com = customer, admin.tankmanpro.com = Admin, partner.tankmanpro.com = Vendor.

Opening a portal hostname while signed in with the wrong role is denied; changing URL/hostname never bypasses authorization.

3. Assumptions and fixed product decisions

This is a greenfield system. Do not assume the deleted AI Studio project or its data/files exist.

Use English UI copy in the first release, with architecture ready for Hindi localization later.

Public storefront must use a light theme. Admin and Vendor panels must also use a clean light theme rather than a terminal-style/dark developer UI.

Admin controls service and vendor prices. Vendors do not set or change the customer-facing price in V1.

Support two pricing/assignment modes per service and city:

City-fixed / auto-assign: customer sees the configured price for that service in that city. The system assigns an eligible vendor according to the routing rule.

Vendor-offer / customer-selects: customer sees eligible vendor offers and each Admin-configured vendor-specific price, and chooses the vendor/offer before checkout.

Effective pricing precedence, where enabled: vendor-service-city override > city-service override > service base price. If a field is unavailable, do not silently invent a price. If a service/city combination has no valid price, stop checkout and show an unavailable/price-not-configured message.

Admin can disable vendor-offer selection and use City-fixed mode for simpler categories.

Launch geography, vendors, prices, coupons, fees, taxes, business rules and contact details are seeded through Admin tools. Sample data must be visibly marked as demo/test data until Admin publishes it.

Razorpay is the planned online payment gateway for India. Keep payment integration disabled until valid server-side secrets and account activation are configured. Offer COD/manual payment only if Admin enables it.

Tax/GST calculation is configurable. Do not invent a tax rate or claim legal compliance; the business owner/accountant must supply the applicable tax settings.

Booking slots and vendor availability must be configurable. Avoid overbooking through server-side transactional checks.

Build one maintainable system and shared backend/database, but provide three separate URL entry points and separate app shells: public customer site, Admin portal and Vendor portal. Production URLs should be https://tankmanpro.com/, https://admin.tankmanpro.com/login, and https://partner.tankmanpro.com/login. The Admin and Vendor portals must not appear as links, tabs, buttons, identity strips or menus anywhere on the public customer website. When custom domains are not configured in AI Studio preview, provide protected same-host development fallbacks at /admin/login and /vendor/login. The URL separation is not a security boundary; enforce role checks in the backend and database rules.

4. Users, roles and permission boundaries

4.1 Guest customer

Can browse public pages and categories, select a service area, view published service information and prices, and begin booking. Authentication may be required before final booking according to configured checkout settings. Guest data cannot access any account or order belonging to another customer.

4.2 Registered customer

Can manage their own profile and addresses, place bookings, view their own bookings, see permitted status updates, cancel when policy permits, download invoices/receipts, apply coupons, contact support and submit a review after an eligible completed booking.

4.3 Main Admin

Full operational access to configuration and business records. Can create/disable Admin-approved vendor accounts; manage locations, service catalog, prices, scheduling, coupons, orders, assignment, payments/refunds (via verified gateway operations), content, reports and audit records. Only trusted Admins can grant roles. Public signup must never allow a visitor to self-select Admin or Vendor.

4.4 Vendor

Can view only bookings whose current assignedVendorId equals their authenticated UID (and whose service area is covered by their active vendor profile). Can accept/reject an assigned booking, add permitted job notes, update work status, and upload allowed job evidence if enabled. Must not see global order lists, other vendors, site-wide reports, coupon rules, gateway secrets, customer payment instruments, arbitrary customer accounts, or Admin settings. Vendor cannot change customer price, discount, payment state or refund status.

4.5 Support / operations role (optional, not required for MVP)

A future restricted role may view support-relevant order details and add internal notes but cannot change prices, payment state, roles or vendor assignment unless separately granted. Do not create this role unless time permits after MVP requirements are complete.

5. Information architecture, separate portal URLs and routes

5.0 URL and app-shell separation (mandatory)

The customer storefront, Main Admin portal and Vendor portal must have different entry URLs. Do not put an Admin/Vendor switcher, dashboard links, developer identity banner, role tabs or portal buttons in the public website header, footer, homepage, service pages or customer account navigation.

Preferred production URLs

Customer storefront: https://tankmanpro.com/

Main Admin portal: https://admin.tankmanpro.com/login

Vendor portal: https://partner.tankmanpro.com/login

AI Studio preview / single-host fallback URLs

Customer storefront: /

Main Admin login: /admin/login

Vendor login: /vendor/login

When custom-domain hostnames are configured, route by hostname to the appropriate app shell. On tankmanpro.com, show only customer-facing pages. On admin.tankmanpro.com, show only Admin portal pages. On partner.tankmanpro.com, show only Vendor portal pages. An unauthorized role must be denied and redirected to the correct login/forbidden state. The hostname or a hard-to-guess URL must never grant privileges.

Use one codebase, one shared backend/API and one authoritative Firestore database for the MVP. Separate frontends/deployments are optional if deployment requires them, but do not duplicate business data. Configure exact allowed origins/CORS, Firebase Authentication authorized domains, password-reset/callback URLs and redirect handling for every configured hostname. Test auth on each hostname independently; browser storage is origin-specific, so do not assume a login cookie/local-storage entry automatically transfers between subdomains.

If DNS/subdomains are not yet configured, keep the fallback routes fully testable in AI Studio while preserving the separate app shells and never linking them from the public storefront. On the production Admin hostname, support portal-local paths such as /login, /dashboard, /orders, /cities, /services, /pricing, and /vendors; on the Vendor hostname, support /login, /dashboard, /orders, /orders/:bookingId, and /profile. The prefixed /admin/* and /vendor/* paths remain the single-host preview fallbacks. If a user opens a prefixed

portal path on the public hostname after custom domains are configured, redirect to the corresponding portal hostname or return a safe not-found response.

5.1 Public/customer routes

/ — Home

/services — service catalog and categories

/services/:slug — service detail, variants, add-ons, price/offer selection

/service-areas — city/PIN code coverage overview

/how-it-works — booking process

/about — brand and trust information

/help — FAQs and support entry points

/contact — contact form and configured business contact information

/login, /register, /forgot-password — authentication

/checkout — booking summary, address, time slot, coupon, payment choice

/booking/success/:bookingId — confirmation only after server-confirmed booking creation

/account — profile overview

/account/addresses — saved addresses

/account/bookings — current and past bookings

/account/bookings/:bookingId — authorized booking detail and cancellation action

/account/reviews — customer's own submitted reviews

/privacy, /terms, /cancellation-refund-policy — editable policy pages

5.2 Vendor routes

/vendor/login

/vendor/dashboard

/vendor/orders

/vendor/orders/:bookingId

/vendor/profile

/vendor/availability — own availability only if this feature is enabled

Vendor routes require a valid authenticated vendor role and server-side authorization on every data request. Redirect other roles to their own portal.

5.3 Admin routes

/admin/login

/admin/dashboard

/admin/orders

/admin/orders/:bookingId

/admin/unassigned-orders

/admin/cities

/admin/service-areas

/admin/categories

/admin/services

/admin/services/:serviceId

/admin/pricing

/admin/vendors

/admin/vendors/:vendorId

/admin/coupons

/admin/slots-and-availability

/admin/customers

/admin/reviews

/admin/content

/admin/reports

/admin/settings

/admin/audit-log

The public customer site must not link to the Admin or Vendor portals in its header, footer, homepage, service pages or account menus. Users who have portal URLs receive them out-of-band from the business owner. Keep each portal's navigation confined to its own hostname/app shell. Never show a role-switcher, active-identity strip or developer shortcut in the public customer UI. Authentication and server-side role authorization remain mandatory even when the user visits the correct hostname.

6. UX/UI and visual design requirements

6.1 Overall direction

Use familiar home-services marketplace usability patterns: clear location selector, category cards, service cards, short booking funnel, transparent pricing, customer trust signals and persistent support entry. The design must be original TankmanPro, not a pixel-level Urban Company reproduction.

6.2 Visual system

Overall theme: white/off-white surfaces, dark navy/charcoal body text, restrained teal accent, subtle grey borders and comfortable whitespace.

Avoid large dark full-page hero backgrounds and "terminal" aesthetics.

Use a modern readable sans-serif font stack and consistent type scale.

Use rounded cards and buttons consistently; shadows should be subtle.

Use real service photography or licensed/AI-generated non-branded illustrations; do not copy competitors' images or logos.

Every image has an appropriate alt text; meaningful images use responsive image sizing.

Use skeletons for loading, clear empty states, readable form errors and non-blocking success/error toasts.

Display the actual city/area in the header once selected and provide a change-location interaction.

Show the price with INR formatting, e.g. ₹799; all price and discount values come from data, never hard-coded into UI.

6.3 Public home page layout

Slim announcement strip only when Admin configures one.

Header with TankmanPro logo/wordmark, location selector, Services, How It Works, Help and Account.

Hero section with concise headline, supporting sentence and prominent six-digit PIN Code field plus “Check availability” CTA.

Popular service category tiles (e.g. Water Tank Cleaning, AC Service, Appliance Repair, Plumbing; only show published categories).

Popular/featured service cards with image, name, starting/effective price, rating only if it is supported by genuine review data, duration estimate when configured, and “View details”.

“How it works” in 3–4 simple steps.

Benefits/trust section with verified claims only. Do not fabricate years in business, number of customers, certification or review counts.

Service area section based on active database records.

Genuine customer testimonials only after real reviews exist; use placeholders labeled demo in development and do not display fabricated reviews as real.

FAQ, support CTA and footer with policies, contact, social links if configured.

6.4 Service listing cards

Each card can show category, service image, service name, short description, effective starting price for selected location, service duration if configured, genuine rating/review count, and booking CTA. If location is not selected and pricing differs by city, show “Select location to see price” instead of misleading price. If service is unavailable in selected PIN code, visibly explain that it is not available there.

6.5 Service detail page

Breadcrumbs and original service imagery.

Service title, overview, inclusions/exclusions, preparation instructions, duration, warranty/guarantee only if Admin configures it, cancellation terms.

Variants/package cards and add-ons with correct pricing.

Location-aware price or vendor offer list, clearly explaining whether a vendor is selected automatically or by customer.

Available date/time slots.

Add coupon/apply coupon on checkout (do not accept or trust a client-calculated discount).

Sticky bottom booking CTA on mobile.

6.6 Responsive behaviour

Mobile-first; support 360 px width and above without horizontal page overflow.

At mobile widths, use compact header, tappable controls (target about 44 × 44 px), 1–2 column card layout as space permits, sticky booking CTA and simple step-based checkout.

Tablet: use a flexible 2–3 column service grid.

Desktop: limit content width for readability, use 3–4 column grids where appropriate, and use persistent sidebar navigation for Admin/Vendor portals.

Tables in dashboards become mobile cards or overflow-safe tables with clear column priorities.

Keyboard navigation, visible focus, labels, meaningful errors, proper contrast and semantic headings are required.

7. Service catalog and data management

Admin can manage all these without changing source code:

Category: name, slug, description, icon/image, display order, active/published flag.

Service: title, slug, category, short/full description, hero/card images, inclusions, exclusions, duration, service mode, base price, active/draft/archived status, SEO title/description.

Variants/packages: name, included scope, effective price rules, duration and active state.

Add-ons: name, description, optional/required, quantity support, price rules and eligible services.

Service attributes: equipment/size/number of units or other category-specific questions. Use dynamic service questions configured by Admin; do not assume every service shares the same fields.

FAQs and policies per service.

Service eligibility per city/PIN code and vendor capability.

Media upload/selection with file validation and size limits.

Archived services cannot be booked for new orders but remain visible in historic order snapshots.

8. City, PIN code and vendor coverage

8.1 Location model

A customer can choose a supported city or enter a 6-digit Indian PIN code. Validate format on client and server. Resolve the PIN to an active service area using the Admin-configured mapping. Do not infer city from a free-text address or trust a client-provided city ID without server validation.

Admin can create cities and service areas, map PIN codes to a city, switch coverage on/off, and see vendors mapped to each area. A PIN code may map to one canonical city/coverage zone for order routing. Support a zone label/neighbourhood field when needed.

8.2 Vendor coverage

Each vendor profile includes display name, legal/business details as needed, contact information, active/pending/suspended status, supported cities/areas, supported service IDs, working days/time windows, capacity/max active jobs (optional), assignment priority and internal notes. Sensitive documents are private and accessible only to authorized Admins.

Vendors cannot self-approve or assign themselves new cities/services. Admin approval is required.

8.3 Unknown/uncovered PIN codes

If the PIN is validly formatted but not serviceable, show a helpful “Not available in this area yet” state. Optionally collect a waitlist request with consent, not a fake booking. Do not create a paid order for an uncovered area.

9. Pricing engine — mandatory business logic

9.1 Pricing data and precedence

Represent prices in integer paise to avoid floating-point currency errors. Store amountPaise and currency: "INR". Effective price resolution:

An explicit vendor + service + city + variant override, if that price level is enabled and valid.

Else a city + service + variant override.

Else the service/variant base price.

If the service/city/vendor combination is invalid or no fallback is allowed, it is not bookable.

All price fields must include effective dates when relevant, active flag, who last changed it and timestamp. Price changes affect new checkouts only. Each booking stores immutable price snapshots (service title, variant, add-ons, unit prices, quantities, subtotal, discount, tax if configured, fees if configured, final total and pricing source).

9.2 Mode A: City-fixed / auto-assignment

Customer sees one clear price for the chosen service, city and variant. Admin configures a city price. After service-area validation and booking/slot checks, system assigns an eligible vendor using the configured strategy: round-robin or least active assigned jobs. Admin may manually override assignment. Vendor-specific internal cost/commission can be stored separately, but must not unexpectedly alter the customer price.

9.3 Mode B: Vendor-offer / customer-selects

For services marked as vendor-offer mode, list only vendors who are active, cover the selected PIN/city, support the selected service, have a valid Admin-configured offer price and can serve the selected slot. Each card must show accurate service price and any supported experience/rating information. Customer picks an offer; the chosen vendor is reserved/assigned to the booking before payment order creation. If that vendor/slot becomes unavailable, invalidate the checkout and ask the customer to choose again. Never silently substitute a different vendor/price after the customer confirmed.

9.4 Pricing admin UI

Admin must be able to: - Set a service base price. - Bulk-edit city prices. - Override price for a specific service in one city. - Override price for a particular vendor, service, city and variant. - Configure which pricing mode is used per service/city. - Preview effective customer price for a sample PIN, service, variant and vendor. - See missing/conflicting/expired prices and prevent publishing a service/city without a valid policy. - Export/import price matrix by CSV with validation and preview before saving.

9.5 Checkout price integrity

Frontend prices are informational. At checkout, backend recalculates from authoritative price records, quantity, add-ons, coupon and configured tax/fees. Require user confirmation if the total changed since the service page. The payment amount is created from this validated server-side total only. Store the final line-item and discount snapshots on the booking.

10. Booking and slot workflow

Customer enters location and verifies coverage.

Customer browses category/service and selects variant/add-ons.

System resolves price mode and shows city price or eligible vendor offers.

Customer selects a date/time slot.

System checks the service's configured duration, slot capacity, active service coverage and vendor availability.

Customer adds/chooses service address, contact details and access instructions.

Customer applies a coupon; the server validates eligibility and recalculates the amount.

Customer reviews complete price breakdown and cancellation/refund policy.

Backend creates a short-lived booking/slot hold to avoid race conditions. If an online payment order is required, create it server-side with amount and booking ID reference.

Customer completes payment through Razorpay Checkout or chooses an Admin-enabled offline payment method.

Backend verifies Razorpay signature and/or receives a valid webhook before confirming paid status. Frontend redirect alone never proves payment.

On verified payment (or approved COD/manual policy), booking becomes confirmed and vendor assignment is finalized. If assignment happened before payment in vendor-offer mode, ensure the selected vendor/slot hold is time-bound and released on expiry/cancel.

Notify customer, assigned vendor and Admin according to notification settings.

Vendor accepts/rejects; accepted work progresses through permitted statuses.

Completion is recorded. Customer may review; Admin handles any dispute/refund.

Use idempotency keys for checkout/payment/webhook processes and transactions/locks where assignment and capacity can race.

11. Coupon and promotion engine

Admin can create/edit/disable coupons with:

Code, internal name, description, active status.

Discount type: percentage or fixed amount.

Discount value; maximum discount cap for percentage coupons.

Start/end timestamps and timezone.

Minimum cart/service amount.

Optional maximum order discount and total redemption limit.

Per-customer redemption limit.

Applicable service IDs, categories, cities/PIN areas, customer segments and pricing modes (all optional filters).

New-customer-only flag (if enabled, define it by verified account/order history).

Exclude specific services/add-ons or combine with other discounts (default: stacking disabled).

Coupon usage count, redemption log and who created/changed it.

Backend must validate the coupon and calculate the discount. Do not trust frontend calculations or client-supplied discountAmount. Coupon eligibility and remaining redemption count must be rechecked atomically when booking is confirmed. Store coupon code and discount snapshot on the booking. Failed/cancelled

payment behavior for coupon usage must be defined and consistently implemented so abandoned attempts do not consume redemption counts permanently.

12. Customer account features

Secure registration/sign-in/sign-out and password reset (Firebase Authentication recommended). Phone OTP is optional and depends on Firebase setup, country configuration and SMS cost; do not fake OTP verification.

Profile name and phone; verify phone if it is used for sensitive notifications.

Multiple saved addresses with label, full address, landmark, PIN code, city and optional lat/long if browser permission is provided.

My Bookings with status, date, service, total and assigned vendor details only to the extent needed by the customer.

Booking detail timeline, reschedule/cancel where allowed, payment receipt/invoice and support request.

Reviews available only for completed eligible bookings; prevent duplicate reviews per booking and moderate in Admin.

Privacy, terms, cancellation/refund policy and consent for optional marketing messages.

13. Vendor portal features

Dashboard cards: New Assigned, Accepted, In Progress, Completed, Cancelled/Rejected (counts based only on that vendor’s permitted data). Main list supports date/status filters and pagination.

Vendor may: - Open only assigned order details. - Accept/reject with a reason from configured options; a rejection sends the order to Admin review or configured reassignment flow. - Update status using allowed transitions only: assigned -> accepted -> in_progress -> completed; reject/cancel paths require the relevant policy and reason. - See necessary customer contact/address and service instructions only for assigned jobs. - Add internal job notes; optional before/after evidence upload can be enabled with private storage rules. - See own schedule and availability if enabled.

Vendor may not: - Search or enumerate global bookings/customers/vendors. - Change booking amount, coupon, payment state or refund state. - Assign orders to themselves or other vendors. - Edit service catalogue, city coverage or another vendor profile. - read gateway secrets, Admin audit log or private documents belonging to others.

14. Main Admin console features

14.1 Dashboard

Useful counts for today and selectable date range: new/awaiting assignment, confirmed, in progress, completed, cancelled, payment exceptions and unassigned. Charts should reflect real data and show an empty state instead of fake metrics.

14.2 Order operations

Search by booking ID, customer, date, city, PIN code, service, vendor, status and payment state. View the complete audit timeline. Assign/reassign vendors only after validating city/service/slot eligibility or explicitly record an Admin override reason. Support cancellation, refund initiation through the gateway when configured, and internal notes with audit history.

14.3 Vendor lifecycle

Create/invite vendor, assign service coverage and price policy, activate/suspend, reset authentication through supported account flows, review job history, and audit role changes. Do not expose passwords in Admin UI or store plaintext passwords.

14.4 Content/settings

Editable homepage banner, category/service display ordering, support email/phone, cancellation rules, booking lead time, slot capacity, checkout methods, currency (INR in launch), tax/fee toggles, terms/privacy text, notification templates and SEO fields. Gateway secrets are never displayed after entry.

14.5 Reports

Filter/export bookings by date/city/service/vendor/status, gross booking amount, discount, tax/fees (when configured), net collected amount based on verified payment data, refunds and vendor performance. Label booking value separately from collected/settled funds. Do not label payment gateway settlement as paid unless backed by gateway data.

15. Payment integration — Razorpay

All Razorpay orders must be created by trusted server-side code.

Browser must not be trusted to specify payable amount or declare a payment successful.

Keep RAZORPAY_KEY_SECRET and RAZORPAY_WEBHOOK_SECRET server-side only through AI Studio's Secrets panel or an appropriate managed secret store. Never commit real values into .env.example, source code, frontend bundles, screenshots or Git history.

Suggested environment configuration names: RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET, RAZORPAY_MODE (test or live), and APP_BASE_URL. Only use names the implementation actually reads.

Use test mode first. Before live use, account activation/KYC, correct live keys, webhook URL, chosen gateway events, refund behavior and successful/failed payment cases must be verified.

Verify checkout signature server-side and webhook signature using the exact raw request body as required by Razorpay's official documentation. Handle duplicate webhook delivery idempotently and log gateway event IDs.

Keep a payment record linked to booking ID: provider, provider order ID, payment ID, amount paise, currency, status, timestamps, verification source and safe error code. Never store full card data or CVV.

Refunds are Admin-initiated only; confirm gateway response and store refund ID/status. Do not mark refunded merely because an Admin button was clicked.

If keys are absent, show "Online payment is not configured" in a controlled manner; no mock success button in production. A clearly labeled local/test payment simulator may exist only in development mode and must be disabled in production.

16. Recommended technical architecture for Google AI Studio

16.1 Stack

Frontend: React + TypeScript using the framework/scaffold generated by Google AI Studio Build mode; use a consistent router, reusable components and responsive CSS.

Design system: Tailwind CSS and a lightweight accessible component library such as shadcn/ui if compatible with the generated scaffold. Avoid adding redundant UI libraries.

Server: Node.js server-side runtime supported by AI Studio; structured API routes/services for admin-only operations, pricing, booking transactions, payment creation and webhooks.

Authentication: Firebase Authentication.

Database: Cloud Firestore with a documented collection schema and indexes.

Authorization: Firebase custom claims or authoritative server-side role records, Firestore Security Rules, and server-side authorization checks on protected endpoints. Do not rely on client route hiding.

Media: Firebase Storage or another managed object store with private rules for vendor job evidence and admin-only files. Public service images should be explicitly public/published.

Payments: Razorpay server-side integration.

Deployment: Google AI Studio Publish to Cloud Run; connect a persistent Firebase project. Export/sync the source to GitHub for version control and backup.

Ask AI Studio to inspect its currently generated scaffold and fit implementation to it rather than inventing incompatible directories. Do not force a WordPress/WooCommerce architecture for this greenfield AI Studio build.

16.2 Backend/API design

Build small, typed service modules rather than one giant server file. Suggested modules:

auth — authentication verification, roles, bootstrap Admin flow

locations — city/PIN code lookup and coverage

catalog — category/service/variant/add-on reads and Admin CRUD

pricing — effective price calculation and price preview

vendors — vendor profile, coverage and availability

slots — schedule availability, holds and expiry

bookings — quote, create, list, status transitions, cancellation

assignment — vendor eligibility, round-robin/least-active logic, reassignment and logs

coupons — validation, discount calculation and redemption usage

payments/razorpay — order create, signature verification, webhook processing, refunds

notifications — email and optional SMS/WhatsApp provider integration

admin/reports — access-controlled aggregations/exports

audit — append-only privileged-action records

Use server-side schema validation for all write requests, proper HTTP status codes, consistent errors and structured logs without exposing secrets or unnecessary customer PII.

16.3 Database collections (proposed)

users/{uid} — profile basics and non-authoritative presentation data; role source must be protected.

cities/{cityId} — city name, state, slug, active status.

serviceAreas/{areaId} — city ID, six-digit PIN code, label, coverage status.

categories/{categoryId} — display fields, sort order, active/published state.

services/{serviceId} — content, category, media, duration, base price, pricing mode/defaults, publishing state.

serviceVariants/{variantId} or service subcollection — variant/package definitions.

serviceAddons/{addonId} or service subcollection — add-ons and eligibility.

vendors/{vendorUid} — Admin-managed business profile and approval state.

vendorCoverage/{coverageId} — vendor UID, city/area IDs, service IDs, active days and availability.

priceOverrides/{priceId} — service, variant, city, optional vendor UID, amount paise, validity and mode.

slotPolicies/{policyId} / slotHolds/{holdId} — slot definitions, capacity and expiry.

bookings/{bookingId} — customer UID, city/area/PIN, service snapshot, selected vendor ID, assignment status, work status, slot, address snapshot, price snapshot, coupon snapshot, timestamps.

payments/{paymentRecordId} — linked booking, gateway IDs, amount paise, verified payment status.

coupons/{couponId} — validation rules and redemption limits.

couponRedemptions/{redemptionId} — customer/booking/coupon and redemption state.

reviews/{reviewId} — booking ID, customer UID, rating, text, moderation status.

notifications/{notificationId} — recipient UID, channel, template key, delivery status.

auditLogs/{logId} — actor UID, action, entity, before/after summary, timestamp, request correlation ID.

siteSettings/{settingId} — non-secret content/settings only.

Do not duplicate large lists of bookings inside a user/vendor document. Prefer queryable collections with appropriate indexes. Snapshot the historic service title, option, price and address on a booking so changes to catalogue data do not rewrite old invoices.

16.4 Authorization model

Public reads only for published categories/services, active service area lookup and public non-sensitive site settings.

Customer can read/update own profile and addresses within allowed fields and read own bookings. Booking creation/price/payment state changes should go through validated backend logic where transactions/business rules are required.

Vendor can read only booking records where assignedVendorId == auth.uid, and may request only allowed status transitions on those bookings. Prefer backend endpoints for state transitions and validate status machine server-side.

Admin routes require server-verified Admin claim/role. Role escalation endpoints must be unavailable to ordinary users.

Firestore rules must default-deny everything not explicitly allowed. Rules must not permit broad reads merely because a user is authenticated.

If Admin SDK/server libraries bypass Firestore rules, secure those server handlers with explicit authentication, authorization and input validation. Test this deliberately.

Write automated tests for cross-vendor access, customer cross-account access, role escalation and coupon/payment manipulation.

16.5 Race conditions and consistency

Vendor assignment, slot capacity, coupon redemption counters and booking/payment transitions must use transactions or idempotent server-side operations where needed.

Implement assignment idempotency so a retried request does not create duplicate assignments or notifications.

Every booking should have a clear assignmentStatus: unassigned, assigned, accepted, rejected, reassignment_required.

Work status and payment status are separate fields with separate allowed transitions.

Use timestamps from server-side code for important business state transitions.

17. Suggested API contracts

Implement equivalent endpoints that fit the generated server framework. Do not put secrets or privileged mutation logic in browser-only code.

Public/customer endpoints

GET /api/public/service-areas?pin=242001 — validate and return coverage status/canonical city.

GET /api/public/catalog?cityId=...&pin=... — published services and effective public prices only.

GET /api/public/services/:serviceId?pin=... — service details and valid pricing choices.

GET /api/public/slots?serviceId=...&pin=...&date=...&vendorId=... — available slot choices.

POST /api/pricing/quote — authoritative quote with service, variant, add-ons, city/PIN, vendor choice and coupon; return breakdown and quote expiry.

POST /api/bookings — create booking/hold using validated quote, address and slot.

GET /api/customer/bookings — current customer's bookings.

GET /api/customer/bookings/:id — only owner can read.

POST /api/customer/bookings/:id/cancel — cancel only when policy permits.

POST /api/payments/razorpay/order — authenticated customer, linked booking, server-calculated amount.

POST /api/payments/razorpay/verify — server-side payment signature verification.

POST /api/webhooks/razorpay — raw-body signature verification, event idempotency.

POST /api/coupons/validate — validate code against server-known cart/location/customer rules; final validation repeats on booking confirmation.

Vendor endpoints

GET /api/vendor/dashboard — counts from vendor-assigned jobs only.

GET /api/vendor/bookings — filter/paginate assigned bookings only.

GET /api/vendor/bookings/:id — assigned-vendor check on server.

POST /api/vendor/bookings/:id/accept

POST /api/vendor/bookings/:id/reject

POST /api/vendor/bookings/:id/status — enforce allowed state transition.

POST /api/vendor/bookings/:id/notes — scoped notes.

Admin endpoints

CRUD for cities/service areas/categories/services/variants/add-ons/vendors/coverage/price overrides/coupons/slots/settings/content.

GET /api/admin/bookings — secure search/filter pagination.

POST /api/admin/bookings/:id/assign — assignment with vendor eligibility validation and reason for override.

POST /api/admin/bookings/:id/cancel

POST /api/admin/payments/:id/refund — provider-confirmed refund flow.

GET /api/admin/reports/bookings — authorized server-side query/export.

GET /api/admin/audit-logs — privileged access only.

All endpoints require typed validation, role checks, authorization by record, safe error messages and logs. Return no private fields unless required for that role.

18. Notification rules

Customer: booking created/confirmed, payment failed where relevant, vendor accepted, rescheduled, cancelled, work completed, refund updates.

Vendor: new assignment, assignment cancelled/reassigned, slot changes and Admin messages.

Admin: unassigned booking, vendor rejection/no response, payment/webhook exception, refund failure and new vendor application if enabled.

Email is the initial channel. SMS/WhatsApp are optional integrations requiring a configured provider, consent where relevant and associated cost approval. Do not simulate a message as delivered when no provider exists.

19. Non-functional requirements

Responsive: 360 px mobile, common Android/iPhone widths, tablets and desktop.

Accessibility: semantic landmarks, proper input labels, keyboard operation, visible focus, contrast, sensible reading order and accessible validation messages.

Performance: optimize images, lazy load below-fold media, avoid unnecessary listeners/reads, paginate lists, avoid oversized bundles and use loading skeletons.

Reliability: clear empty/error/loading states; retry safe/idempotent operations; no duplicate booking or payment record after a retry.

Security: least privilege, verified identity, server-side role checks, input validation, output escaping, rate limiting for sensitive endpoints, secure file access, no secrets in client bundle/logs.

Privacy: collect only necessary customer data, restrict PII to authorized staff/vendor assigned jobs, and provide editable privacy/retention policies.

Maintainability: typed models, small reusable modules, shared design components, environment example file with placeholders only, README and deployment instructions.

Observability: safe structured logs, correlation IDs for payment/booking issues, audit trail for privileged changes. Do not log passwords, key secrets, full payment data or excessive PII.

SEO: readable slugs, page titles/descriptions, canonical URLs where needed, sitemap/robots support and server/deployment constraints documented. Public service pages should be crawlable where supported by framework.

20. Security requirements and prohibited shortcuts

Never hard-code production secrets or put Razorpay Secret/Webhook Secret into client-side code.

Never trust amount, discount, city, vendor ID, payment success, role or booking status supplied by the browser without authoritative server validation.

Never allow public signup to create Admin or Vendor roles.

Never rely only on hidden menu items for authorization.

Never query all bookings then filter only in the browser for a vendor.

Never ship Firestore rules that allow unrestricted read/write for all authenticated users.

Never show fictitious ratings, testimonials, certificates, service counts or successful payments as real.

Never deploy test payment simulation as a live payment method.

Do not call a webhook secure unless raw-body signature verification and duplicate-event handling are implemented and tested.

Do not claim production readiness until all P0 acceptance tests pass.

21. Admin first-run setup

The first Admin must be provisioned through a controlled setup process, not ordinary public signup. Implement one of these approaches and document it clearly:

A one-time server-side bootstrap that only grants the Admin role to a preconfigured, verified email/UID while no Admin exists, then permanently disables itself; or

A trusted Firebase/Google Cloud Admin SDK script executed by the project owner to assign the custom claim and protected user role.

Do not make INITIAL_ADMIN_EMAIL alone a public privilege-escalation mechanism. No secret or role-grant token should be sent to the browser. Document how to create, verify and recover the first Admin account.

22. Seed data and demo mode

Provide a controlled seed process for demo categories/services such as Water Tank Cleaning, AC Service, Washing Machine Repair, Refrigerator Service and Plumbing. Example city records can include Delhi and Shahjahanpur; PIN codes such as 110001 and 242001 may be used as sample test inputs only. Add an explicitly uncovered test PIN such as 400001 only if the seed script guarantees it is not mapped to an active area. These are test examples, not a claim of actual coverage.

Seed/demo prices, vendor names, reviews and counts must carry a demo marker and must not be represented as real customers, rates, ratings or approved vendors. Admin should be able to remove all demo data before launch. Never seed fake Razorpay success events.

23. Testing and acceptance criteria

P0 — release blockers

Coverage: active configured PIN returns the correct city and available services; unknown/uncovered PIN cannot complete a paid booking.

City pricing: changing city changes the quote according to configured rules, not a hard-coded front-end constant.

Vendor pricing: Admin can set a vendor/service/city price override and quote returns the correct effective price.

Customer-selected offer: in vendor-offer mode, selected vendor and price remain tied to the booking; checkout cannot silently switch vendor.

Auto assignment: in city-fixed mode, assignment selects only a vendor that is active, covers the city/area, supports the service and can serve the slot.

No vendor available: order is not falsely confirmed as assigned; Admin gets an unassigned alert/state.

Vendor isolation: Vendor A cannot list, read, update or infer Vendor B's bookings by changing URL, query parameters, document ID or API payload.

Customer isolation: Customer A cannot access Customer B's profile, address, booking or invoice.

Role escalation: public user cannot promote self to Admin/Vendor by editing Firestore fields or request body.

Coupon validation: invalid, expired, non-eligible, over-limit and city/service-restricted coupons are rejected server-side.

Price integrity: browser-tampered subtotal/discount/vendor/city values do not change the final server-calculated amount.

Razorpay: no paid status from client redirect alone; invalid signature, duplicate webhook and mismatched amount are rejected or safely idempotent.

Slot contention: concurrent booking attempts cannot overbook capacity for the same slot.

Vendor state changes: invalid status transitions are rejected and logged.

Responsive QA: all key customer, admin and vendor screens work at mobile, tablet and desktop breakpoints.

P1 — important after P0

CSV import/export with preview and row validation for service pricing.

Reviews moderation and customer review after completed booking.

Email notification templates and delivery error state.

Admin reports by date/city/service/vendor.

Customer reschedule flow with slot revalidation.

Vendor job evidence upload with private access policy.

Accessibility audit and performance pass.

QA scenarios

Use seeded test accounts: one Admin, two customers, and at least two vendors in different cities plus two vendors in one city for round-robin/offer selection. Verify every P0 scenario. Use Razorpay test mode and a webhook test tool/test event. Do not use real payment credentials during initial functional QA.

24. Build, version control and deployment using Google AI Studio

Open Google AI Studio Build mode and start a brand-new web application. Do not try to restore or reference the deleted project.

Upload/provide this PRD as context where the interface allows it, then paste the Master Prompt from the companion prompt file.

Ask the agent to first summarize architecture, risks, collection schema and build phases; then implement Phase 1 only. This reduces the risk of receiving a giant untested prototype.

Enable Firebase integration for Authentication and Firestore. Carefully choose project and region/location before confirming, because Firebase/Google Cloud resources use the selected setup. Keep Firestore in locked/deny-by-default rules and add rules deliberately.

Develop the backend/server modules, payment integration and authorization before declaring the checkout complete.

After each phase, run type checks/build/tests and inspect Preview at mobile and desktop sizes. Fix failures before continuing.

Add secrets through the AI Studio Secrets UI/server-side configuration. Use fake placeholders in .env.example; never store actual secrets in Git.

Sync/export to GitHub regularly. Use commits/checkpoints before risky changes.

Publish to Cloud Run only after tests pass. Configure a custom domain, HTTPS, Firebase authorized domains, Razorpay webhook, production secrets, backups and error monitoring.

Conduct the full end-to-end acceptance suite after deployment using test mode, then switch to live payment only after the business owner’s payment account is approved and production configuration verified.

25. Phased implementation plan

Phase 0 — Architecture and scaffold

Confirm current AI Studio scaffold, dependencies, app routing, design tokens, server entry points, Firebase plan, collection schema and environment variable names. Create a readable project structure, README and typed domain models. Do not connect live payments yet.

Phase 1 — Original public UI

Build the responsive TankmanPro visual system, homepage, location selector, service categories/cards, service details shell, policies and help/footer. Use demo content labelled as demo. No fake data persistence claims.

Phase 2 — Firebase and roles

Set up Firebase Authentication, Firestore, rules, indexes, role model, secure first Admin provisioning, customer profile and protected route behavior. Add rules/emulator tests.

Phase 3 — Admin CRUD

Build CRUD for cities, service areas, categories, services, variants, add-ons, vendors, vendor coverage and pricing overrides. Validate on server and add empty/loading/error states. Customer-facing data should use published records only.

Phase 4 — Pricing and coupons

Implement effective-price resolver and both pricing modes, quote API, coupon eligibility/limits, price snapshots and unit tests. Do not move on until server-calculated totals are correct.

Phase 5 — Booking and vendor assignment

Implement slots, slot holds, booking create/cancel/reschedule (if scoped), assignment strategy, manual reassignment, assignment history and vendor work status state machine. Add isolation and concurrency tests.

Phase 6 — Payments and notifications

Integrate Razorpay in test mode using server secrets. Implement payment order, signature verification, webhook raw-body verification, idempotent event processing, refunds and notification provider abstraction. Keep online payment disabled until configured.

Phase 7 — Admin/Vendor analytics and polish

Build reports, customer booking history, invoice/receipt, reviews and moderation, responsive dashboard polish, accessibility, SEO and performance optimizations.

Phase 8 — Launch readiness

Run every P0 acceptance test, security review, Firestore rules tests, secret scan, mobile/desktop QA, broken-link checks, and payment test cases. Create deployment/recovery documentation and obtain owner approval before switching to live payments.

26. Definition of done

The project is not done because Preview looks attractive. It is done when:

All P0 criteria pass.

The app builds without TypeScript/build errors.

Firebase Auth/Firestore are persistent, and rules are explicitly configured and tested.

All Admin CRUD operations persist after reload.

Service prices/coupons are calculated by the server and show a correct breakdown.

Customer can complete the booking flow with a supported payment option in test mode.

Vendor only accesses their own assigned order data.

Admin can inspect and reassign orders, manage all business data and see audit logs.

No secrets, debug identity banners, fake ratings or developer placeholders remain in production UI.

Responsive review completed for public pages, checkout, Admin and Vendor portals.

README explains local/AI Studio setup, Firebase rules, Admin bootstrap, test users, secrets, payments and deployment.

27. Official references for implementation

Use current official documentation during implementation; verify UI labels and available settings at build time.

Google AI Studio Build mode: https://ai.google.dev/gemini-api/docs/aistudio-build-mode

Deploy from Google AI Studio: https://ai.google.dev/gemini-api/docs/aistudio-deploying

Add Firebase to a Google AI Studio app: https://firebase.google.com/docs/ai-assistance/ai-studio-integration

Firestore Security Rules: https://firebase.google.com/docs/firestore/security/get-started

Cloud Run secrets/configuration: https://docs.cloud.google.com/run/docs/configuring/services/environment-variables

Razorpay documentation: https://razorpay.com/docs/

Appendix A — Required screens checklist

Customer

☐ Home; location modal/drawer; service search; category and service listing.

☐ Service detail; variant/add-on selection; vendor offer selection where configured.

☐ Date/time selector; address selector/add address; coupon and price breakdown.

☐ Payment choice; payment processing, failure and success states.

☐ Booking confirmation; booking history; booking detail and timeline.

☐ Profile; saved addresses; reviews; help/contact; policies.

Admin

☐ Login; overview dashboard; order list/detail; unassigned queue.

☐ Cities; service areas/PIN codes; categories; services; variants; add-ons.

☐ Pricing matrix and effective price preview; coupon CRUD.

☐ Vendors; vendor coverage/service mapping; availability/capacity.

☐ Customers; reviews/moderation; content/FAQ/policy editor.

☐ Reports/export; payment/refund exception queue; settings; audit log.

Vendor

☐ Login; own dashboard; assigned order list; allowed filters.

☐ Assigned order detail; accept/reject with reason; job status update.

☐ Own schedule/availability if enabled; own profile; private job note/evidence upload if enabled.

☐ Denied states for unassigned/other-vendor order IDs.

Appendix B — Example price calculation

Illustrative only: base price ₹1,000; Shahjahanpur city override ₹900; selected vendor override ₹850. A valid 10% coupon gives an ₹85 discount, subject to configured caps and rounding. Apply only Admin-approved taxes/fees, store amounts in paise, and snapshot the calculation on the booking. These are not live TankmanPro prices.

Appendix C — Recommended default business settings

Default booking mode: city-fixed + auto-assign until Admin enables vendor-offer mode for a service/city.

Default assignment: least-active eligible vendor with round-robin tie-breaker; Admin can choose another strategy.

Price source: explicit vendor-city-service override, then city-service override, then base price, subject to enabled pricing mode.

Coupon stacking: off by default.

Uncovered PIN: no booking, no payment; show coverage message and optional waitlist request.

Defaults: assigned jobs only for vendors; Admin handles refunds; payment tests only; reviews moderated.